

AngularJS Advanced Examination

MCA 3rd Semester

Modules Covered: 1–8 (Introduction & Architecture → Performance & Best Practices)

Total Questions: 40 | **Format:** Multiple Choice | **Target Difficulty:** 9.5/10

Instructions for students: Every option below is written to be technically plausible. Surface familiarity with a keyword is not enough — you must trace execution order, scope inheritance, and AngularJS's internal digest/DI mechanics precisely. Assume `app` refers to an already-defined module (`angular.module('app', [...])`) in every code snippet unless stated otherwise. A score above 80% reflects genuine practical mastery, not memorized definitions.

Section A: Scenario-Based Questions (Q1–Q10)

Q1. A developer builds an SPA with a parent controller `ParentCtrl` where `$scope.username = 'admin'`, and a nested `<div ng-controller="ChildCtrl">` whose template contains `<input type="text" ng-model="username">`. `ChildCtrl` itself never touches `username`. When the user types into that input, a `{{username}}` binding rendered elsewhere under `ParentCtrl` does not update. What is the root cause?

- ☐ A) `ng-model` does not support two-way binding across nested controllers
- ☐ B) The child scope, via prototypal inheritance, gains its own own primitive `username` property the moment the user types (a JS property write never "writes through" to the prototype), so it shadows the parent's copy instead of updating it
- ☐ C) AngularJS requires `$apply()` to be called manually for `ng-model` to propagate values to an ancestor scope
- ☐ D) `ChildCtrl` needs an isolate scope declared in order to correctly inherit primitive values from its parent

Q2. A `$watch` on `$scope.stats` recalculates a derived value and, inside its own listener, mutates a property of the very object it is watching, on every change. In production, the tab occasionally freezes and the console prints `10 $digest() iterations reached. Aborting!`. What is the most likely cause?

- ☐ A) The `$watch` listener modifies data that the same (or another) watcher is monitoring, so the model never stabilizes within AngularJS's fixed digest-loop limit
- ☐ B) `$scope.stats` is too large an object for AngularJS to digest, exceeding a hidden internal size limit
- ☐ C) The developer forgot to inject `$rootScope`, so the digest loop has no terminating condition
- ☐ D) `$watch` listener callbacks execute asynchronously in AngularJS 1.x, causing a race condition against `$digest`

Q3. A developer integrates a third-party charting library. Inside the library's native `onZoom` callback (fired by the library's own internal event system, not by any Angular-aware API), the developer assigns `$scope.zoomLevel = newLevel`. The `{{zoomLevel}}` binding in the template never refreshes, even though a `console.log` inside the callback confirms the scope variable changed. What should the developer do?

- ☐ A) Wrap the assignment in `$scope.$apply(function() { $scope.zoomLevel = newLevel; })` so AngularJS knows a digest cycle is needed
- ☐ B) Replace `{{zoomLevel}}` with `ng-bind`, since interpolation cannot reflect externally-triggered variable changes
- ☐ C) Move only the chart library's initialization call inside a `$timeout` block, reasoning that anything wrapped in `$timeout` once becomes permanently digest-aware for its entire lifecycle
- ☐ D) Use `$scope.$eval('zoomLevel = ' + newLevel)` instead of direct assignment, since `$eval` is parsed by Angular's expression compiler

Q4. A shared module must let downstream applications configure a `baseApiUrl` inside a `.config()` block — before any controller, service, or factory is instantiated. Which recipe type is required to expose this configuration hook, and why?

- A) `.factory()`, because factories are AngularJS's most flexible recipe type and can accept configuration objects passed in advance
- B) `.provider()`, because only providers are available for injection during the config phase, before the injector has finished being built and before any other recipe's `$get` function has run
- C) `.service()`, since services are constructor functions, and constructors are always instantiated before other recipe types during bootstrap
- D) `.constant()`, because constants are available during the config phase and, unlike other recipe types, can still be reconfigured afterward by other services during the run phase

Q5. A product listing uses `<li ng-repeat="p in products | priceFilter:selectedCurrency | orderBy:'price'">`, where `priceFilter` performs a currency-conversion calculation. As unrelated `$watch` expressions accumulate elsewhere on the page, typing into a completely unrelated search box starts to visibly lag. What explains this, and what is the best fix?

- A) `priceFilter` re-executes across the entire `products` array on every single digest cycle, regardless of whether `products` or `selectedCurrency` actually changed, because filters run on each digest; the fix is to precompute/cache converted prices in the controller rather than recompute them in the template on every cycle
- B) `orderBy` is deprecated in favor of a controller-side comparator function, and using it causes the observed lag
- C) `ng-repeat` cannot process more than one piped filter without significant performance loss; filters must always be applied only inside the controller
- D) The search box triggers a `$rootScope.$broadcast`, which forces `priceFilter` to re-run recursively across every sibling scope in the app

Q6. A custom directive `myTooltip`, used twice on the same page, defines no scope property at all (not `true`, not `{ }` — simply omitted). QA reports that hovering over the second tooltip instance sometimes changes the text displayed by the first tooltip's internal state. What is the architectural mistake?

- A) Without any isolate scope, the directive uses whatever scope already exists at that point in the DOM (often the very same shared/inherited scope for both instances), so an internal flag set inside one instance's link function (e.g. `scope.activeText = . . .`) leaks across instances reading from that identical shared scope object
- B) AngularJS directives are singletons by default, so both `myTooltip` elements are forced to reference one identical DOM node in memory
- C) The text attribute was not wrapped in `{{ }}` interpolation, causing both directive instances to reference one static string
- D) Multiple instances of the same custom directive are only supported inside `ng-repeat`; using raw duplicate markup is unsupported

Q7. An app works flawlessly in local development but throws `Unknown provider: eProvider <- e` only in the minified production build. What is the underlying cause?

- A) Minifiers rename function parameter names, and AngularJS's implicit DI annotation resolves dependencies by reflecting on those exact parameter names via `Function.prototype.toString()` — once renamed, the names no longer match any registered service; the fix is `array` or `$inject` annotation
- B) A dependency was never registered on the module, and minification merely exposed a pre-existing latent bug
- C) Minified code cannot execute `$http` calls, because `$http` requires unminified property names to construct requests
- D) The controller must be rewritten using `controller` as syntax, which is inherently minification-safe by design

Q8. A dashboard controller references `$scope.userProfile.name` during its own initialization logic, but `userProfile` is only populated after an asynchronous `$http` call finishes inside that same controller. Users briefly see a console error on page load before data arrives. What is the recommended architectural fix?

- ☐ A) Wrap the initialization logic in a `$timeout` with a fixed delay (e.g. 2000ms) chosen to comfortably outlast the expected HTTP response time
- ☐ B) Use the route's `resolve` property to fetch `userProfile` before the controller is instantiated at all, so the controller only ever runs once the data already exists
- ☐ C) Move the `$http` call into a module-level `.run()` block so it executes before any route is matched
- ☐ D) Wrap the entire view template in `ng-if="userProfile"` to hide rendering errors until the data loads

Q9. A deeply nested child controller detects an expired session and must notify every ancestor scope, including `$rootScope`, so a separate, unrelated navigation-bar controller elsewhere in the app (which listens on `$rootScope`) can show a "Session Expired" banner. Which method should the child use, and why?

- ☐ A) `$scope.$broadcast('sessionExpired')`, because broadcast notifies every scope in the entire application regardless of hierarchy
- ☐ B) `$scope.$emit('sessionExpired')`, because emit propagates an event upward through every ancestor scope up to and including `$rootScope`, and any listener registered along that upward chain will catch it
- ☐ C) `$rootScope.$watch('sessionExpired', ...)`, because watchers can directly observe custom event names
- ☐ D) `$scope.$apply('sessionExpired')`, because `$apply` accepts an event-name string and re-runs the digest scoped to that event

Q10. An SPA registers `$rootScope.$on('userUpdated', callback)` inside every instance of a frequently-reloaded child controller (instantiated dozens of times per session as the user navigates). QA reports the app becomes progressively slower, and eventually a single broadcast event appears to trigger the same handler logic multiple times. What is the root cause and fix?

- A) `$rootScope` listeners are automatically garbage collected the moment the registering controller's scope is destroyed, so this must be a bug inside `$broadcast` itself
- B) Because the listener is registered on `$rootScope` — which never gets destroyed — rather than on the controller's own `$scope`, each new controller instance adds one more surviving listener that is never cleaned up; fix by registering on `$scope.$on` instead, or by explicitly calling the deregistration function `$rootScope.$on` returns inside a `$scope.$on('$destroy', ...)` handler
- C) `$broadcast` queues and replays every historical event whenever a new listener is added; this is expected AngularJS behavior
- D) The fix is to replace `$rootScope.$on` with `$rootScope.$watch`, since watchers (unlike listeners) are automatically cleaned up on every route change

Section B: Code Analysis Questions (Q11–Q20)

Q11.

```
app.controller('DemoCtrl', function($scope) {  
  $scope.counter = 5;  
  var callCount = 0;  
  
  $scope.$watch('counter', function(newVal, oldVal) {  
    callCount++;  
    console.log('Fired. Count:', callCount, 'New:', newVal, 'Old:', oldVal);  
  });  
});
```

Assuming counter is never changed by any other code, and exactly one digest cycle runs after registration (e.g., triggered once during bootstrap), what is logged?

- ☐ A) Nothing — since counter never changes, the watcher's listener never fires at all
- ☐ B) Fired. Count: 1 New: 5 Old: 5 — the listener always fires at least once on the very next digest after registration, with newVal and oldVal both equal to the initial value
- ☐ C) Fired. Count: 1 New: 5 Old: undefined — because oldVal is always undefined the first time any watcher's listener runs
- ☐ D) An error is thrown, because \$watch requires the watched expression to change at least once before a listener can be registered

Q12.

```
<div ng-controller="ListCtrl">
  <div ng-repeat="item in items">
    <input type="checkbox" ng-model="isChecked"> {{item.name}}
  </div>
  <p>Parent's isChecked: {{isChecked}}</p>
</div>

app.controller('ListCtrl', function($scope) {
  $scope.items = [{name:'A'}, {name:'B'}, {name:'C'}];
  $scope.isChecked = false;
});
```

The developer intended one shared boolean to control all three checkboxes and the paragraph together. Instead, when the user clicks only the second checkbox, just that one checkbox becomes checked — the other two, and the parent paragraph, still show unchecked/false. Why?

- A) `ng-model` cannot bind boolean values inside `ng-repeat`; only strings or object properties are supported there
- B) `isChecked` is a bare primitive, and each `ng-repeat` iteration owns a child scope that prototypally inherits it; clicking a checkbox performs a write, and in JavaScript a property write always creates a new own property on the scope object performing it rather than writing through the prototype chain — so only that one iteration's child scope is affected, while the other iterations and the parent (who never had their own copy touched) still read the original unchanged value
- C) `ng-repeat` shares one single scope object across all iterations by default, so this must be a digest-cycle race condition rather than a scope-inheritance issue
- D) The parent's paragraph sits outside the `ng-repeat` element, so it is compiled before the repeated block exists and its binding is permanently frozen at its initial value

Q13.

```

app.directive('outer', function() {
  return {
    restrict: 'E',
    compile: function() {
      console.log('outer compile');
      return {
        pre: function() { console.log('outer preLink'); },
        post: function() { console.log('outer postLink'); }
      };
    }
  };
});
app.directive('inner', function() {
  return {
    restrict: 'E',
    compile: function() {
      console.log('inner compile');
      return {
        pre: function() { console.log('inner preLink'); },
        post: function() { console.log('inner postLink'); }
      };
    }
  };
});

<outer><inner></inner></outer>

```

What is the exact console output order?

- ☐ A) outer compile, inner compile, outer preLink, inner preLink, inner postLink, outer postLink
- ☐ B) outer compile, outer preLink, inner compile, inner preLink, inner postLink, outer postLink
- ☐ C) outer compile, inner compile, outer preLink, outer postLink, inner preLink, inner postLink
- ☐ D) inner compile, outer compile, inner preLink, outer preLink, outer postLink, inner postLink

Q14.

```
app.controller('TimerCtrl', function($scope, $timeout) {  
    $scope.message = 'Waiting...';  
  
    setTimeout(function() {  
        $scope.message = 'Updated via setTimeout';  
    }, 1000);  
  
    $timeout(function() {  
        $scope.message = 'Updated via $timeout';  
    }, 2000);  
});
```

Assuming nothing else triggers a digest during this window (no ng-click, no \$http, etc.), what does the user actually see rendered in {{message}} by the 2.5-second mark?

- ☐ A) 'Updated via setTimeout' appears at 1 second, then 'Updated via \$timeout' appears at 2 seconds — both render normally since both functions run asynchronously in the same way
- ☐ B) Still 'Waiting...' the entire time, since neither callback fires without a manual \$apply call somewhere
- ☐ C) The view stays on 'Waiting...' from 0–2 seconds (the native setTimeout callback silently changes the underlying scope value at 1 second, but nothing triggers a digest to reflect it), then jumps directly to 'Updated via \$timeout' at 2 seconds when \$timeout's own built-in digest finally fires and picks up whatever the current scope value is by then
- ☐ D) A runtime error is thrown at 1 second, because native setTimeout is disallowed inside an AngularJS controller

Q15.

```

app.directive('userCard', function() {
  return {
    restrict: 'E',
    scope: { userName: '@', userData: '=', onSelect: '&' },
    template: '<div>{{userName}}<button ng-click="onSelect()">Pick</button></div>',
    link: function(scope) {
      scope.userData.visits = (scope.userData.visits || 0) + 1;
    }
  };
});

<user-card user-name="{{currentUser.name}}" user-data="currentUser" on-
select="selectUser(currentUser)"></user-card>

```

Given `$scope.currentUser = {name: 'Riya', visits: 3}` in the parent controller, what is `$scope.currentUser.visits` in the parent's own scope immediately after the directive links?

- ☐ A) Still 3 — isolate scope directives are fundamentally prevented from mutating any object owned by the parent scope
- ☐ B) 4 — `userData: '='` creates a genuine two-way binding to the exact same object reference; mutating `scope.userData.visits` inside the directive mutates the identical object the parent also points to
- ☐ C) undefined — `'='` bindings in AngularJS 1.x only flow one-way, from parent to child
- ☐ D) NaN — `visits` arrives as an interpolated string via `'@'` binding and cannot be incremented numerically

Q16.

```
app.controller('ApiCtrl', function($scope, $http) {  
  $scope.users = [];  
  $http.get('/api/users')  
    .then(function(response) { $scope.users = response.data; })  
    .catch(function(error) { console.log('Failed:', error.status); });  
});
```

The server returns HTTP 200 with the JSON body { "data": [...] } — i.e., the server's own payload happens to use a top-level key literally named data. What does `$scope.users` end up holding?

- ☐ A) The inner array of users directly, because AngularJS automatically unwraps nested data keys for convenience
- ☐ B) The entire parsed response body object, { data: [...] } — because `$http`'s response object always wraps the server's raw payload inside its own `.data` property regardless of what keys that payload itself contains; `response.data` here equals the server's whole {data: [...]} object, not the inner array
- ☐ C) undefined — a payload containing a key literally named data collides with `$http`'s internal `.data` property and silently fails
- ☐ D) The `.catch()` block runs instead, since a data-key collision is treated by `$http` as a malformed response

Q17.

```
<li ng-repeat="p in products | filter:{inStock:true} | orderBy:'-price' |
  limitTo:3">
  {{p.name}} - {{p.price}}
</li>
```

What does AngularJS actually render?

- ☐ A) The 3 highest-priced in-stock products, sorted highest to lowest — filters apply strictly left to right: first reduce to in-stock items, then sort descending by price, then take the first 3 of that already-sorted, already-filtered result
- ☐ B) Exactly 3 random in-stock products, since `limitTo` selects arbitrary elements once combined with `orderBy`
- ☐ C) The 3 lowest-priced products overall, stock status ignored, because `limitTo` is silently applied first regardless of its position in the pipe
- ☐ D) All in-stock products sorted by price descending, with `limitTo:3` simply ignored, since `ng-repeat` cannot chain more than two piped filters

Q18.

```
app.factory('CartService', function() {
  var items = [];
  return {
    add: function(item) { items.push(item); },
    getCount: function() { return items.length; }
  };
});
app.controller('CtrlA', function($scope, CartService)
{ CartService.add('Book'); });
app.controller('CtrlB', function($scope, CartService) { $scope.count =
CartService.getCount(); });
```

CtrlA runs and adds an item before the user navigates to the view controlled by CtrlB. What does `$scope.count` hold in CtrlB?

- ☐ A) 0 — each controller receives its own freshly-instantiated copy of `CartService`
- ☐ B) 1 — factories are singletons within a single injector; both controllers receive references to the exact same `CartService` instance, and therefore the same closed-over `items` array
- ☐ C) undefined — `items` is a private closure variable and cannot be reached indirectly through methods called from a different controller
- ☐ D) It depends on whether `$rootScope.$apply()` ran between the two controllers loading

Q19.

```
<form name="signupForm" novalidate>
  <input type="email" name="email" ng-model="user.email" required>
  <div ng-if="signupForm.email.$invalid && signupForm.email.$dirty"
class="error">
    Please enter a valid, non-empty email.
  </div>
  <button ng-disabled="signupForm.$invalid">Submit</button>
</form>
```

On first page load, before any interaction, the error `<div>` correctly stays hidden while the Submit button is (also correctly) disabled. A developer then changes the error condition to `ng-if="signupForm.email.$invalid"`, removing `&& signupForm.email.$dirty`, while leaving the button untouched. What is the immediately visible result, and which `ngModelController` state was responsible for preventing it before the edit?

- A) The error message now shows immediately on fresh page load, because `email.$invalid` is already true before any typing (an empty required field is invalid by definition); previously, the `$dirty` check — the inverse of `$pristine`, which starts true and flips only once the user actually modifies the field — suppressed the message until genuine interaction occurred. The button's own state was never tied to `$dirty`, so it remains correctly, independently disabled either way
- B) Nothing changes visually, since `$invalid` and `$dirty` always hold identical values for any given field
- C) The form fails to compile, because `ng-if` cannot reference nested form-control properties such as `signupForm.email.$invalid`
- D) The Submit button becomes permanently enabled after this edit, because removing a condition from one element in a form removes it from every other element in that same form

Q20.

```
<div ng-if="showPanel">
  <div ng-controller="PanelCtrl"></div>
</div>

app.controller('PanelCtrl', function($scope) {
  console.log('PanelCtrl instantiated');
});
```

The user triggers three sequential toggles of `$scope.showPanel`, in this order: `false`→`true`, then `true`→`false`, then `false`→`true` (ending on `true`). How many times does 'PanelCtrl instantiated' print in total, and why?

- ☐ A) Once total — AngularJS caches controller instances by name for performance
- ☐ B) Twice — `ng-if` only re-renders when the DOM was previously removed and the scope was separately flagged dirty by a digest, a condition that happens to hold on alternating toggles only
- ☐ C) Twice — `ng-if` completely destroys the element, its scope, and the nested controller whenever the expression becomes falsy, and creates a brand-new element/scope/controller only each time the expression subsequently becomes truthy again; here it transitions from falsy to truthy exactly twice (the initial `false`→`true`, and the final `false`→`true`)
- ☐ D) Three times — every toggle of `showPanel`, whether it becomes truthy or falsy, forces `ng-if` to instantiate a fresh controller in order to correctly re-evaluate the expression

Section C: Debugging Questions (Q21–Q30)

Q21.

```

app.directive('confirmBox', function() {
  return {
    restrict: 'E',
    transclude: true,
    scope: { message: '@' },
    template: '<div class="box"><p>{{message}}</p><div ng-transclude></div></div>',
    link: function(scope) {
      scope.username = 'Guest'; // intended to be visible inside the
      transcluded content
    }
  };
});

<div ng-controller="PageCtrl">
  <confirm-box message="Please confirm">
    <p>Hello, {{username}}!</p>
  </confirm-box>
</div>

app.controller('PageCtrl', function($scope) { $scope.username = 'Aditi'; });

```

The developer expects the transcluded paragraph to render Hello, Guest!, since `scope.username = 'Guest'` was set inside the directive's own link function. It instead renders Hello, Aditi!. What explains this?

- A) Transcluded content is always compiled and linked against the original scope from the DOM location where it was authored — here, PageCtrl's scope — never against the directive's own isolate scope; `{{username}}` inside `<confirm-box>...</confirm-box>` therefore resolves against `PageCtrl.username`, completely unaffected by anything the directive's link function does to its own isolate scope
- B) This is a genuine AngularJS bug; transcluded content should always use the directive's isolate scope, and this output means the directive is missing a `scope: false` declaration
- C) `{{username}}` is evaluated against both scopes simultaneously, and AngularJS displays whichever assignment happened most recently in wall-clock time

- D) `ng-transclude` ignores scope variables set inside a directive's link function, but would correctly pick them up if they were instead set inside the directive's compile function

Q22.

```
app.controller('ProfileCtrl', function($scope) {
  $scope.username = 'Alex';
  $scope.updateName = function(newName) { $scope.username = newName; };
});

<div ng-controller="ProfileCtrl as profile">
  <p>{{profile.username}}</p>
  <input ng-model="profile.username">
</div>
```

The paragraph never displays 'Alex', and typing in the input has no visible effect on it. What is the bug and the fix?

- A) `ng-model` cannot be combined with `controller as` syntax under any circumstances; it must be replaced with `ng-bind`
- B) The controller assigns properties directly onto `$scope` (`$scope.username`), but the template references the controllerAs alias `profile.username`; `controller as` syntax expects data to live on `this` inside the controller (`this.username = 'Alex'`), so the two never connect — fix by using `this.username` throughout the controller (or drop the `as profile` alias and bind directly to `username`)
- C) The alias name `profile` is a reserved AngularJS keyword and cannot be used with `ng-controller as`
- D) `ng-controller="ProfileCtrl as profile"` is invalid syntax; the correct form is `ng-controller="ProfileCtrl" controller-as="profile"`

Q23.

```

app.directive('ratingWidget', function() {
  return {
    restrict: 'E',
    scope: { score: '=' },
    template: '<span>{{score}} / {{maxRating}}</span>',
    link: function(scope, element, attrs) {
      // maxRating is expected to come from the parent controller's
      $scope.maxRating
    }
  };
});

<div ng-controller="ProductCtrl">
  <rating-widget score="product.rating"></rating-widget>
</div>

app.controller('ProductCtrl', function($scope) {
  $scope.product = { rating: 4 };
  $scope.maxRating = 5;
});

```

{{score}} correctly displays 4, but **{{maxRating}}** renders blank. What is the bug?

- ☐ A) maxRating was never declared inside the directive's isolate scope: {} bindings, so the isolate scope has no access whatsoever to the parent's \$scope.maxRating — unlike a normal prototypally-inheriting child scope, an isolate scope only exposes properties it explicitly declares as bindings
- ☐ B) maxRating must be written as the attribute max-rating in the HTML for AngularJS to detect it automatically, even without declaring it in scope: {}
- ☐ C) Isolate scope templates cannot render more than one interpolated expression in a single template string
- ☐ D) The link function must explicitly call scope.\$apply() for any binding beyond the first one to render

Q24.

```
app.config(function($routeProvider) {
  $routeProvider
    .when('/user/:userId', { templateUrl: 'user.html', controller:
    'UserController' })
    .otherwise({ redirectTo: '/' });
});

app.controller('UserController', function($scope, $routeParams) {
  $scope.id = $routeParams.userId;
});
```

Navigating to `#/user/42` results in `$scope.id` being undefined. What is the bug?

- ☐ A) Route parameter keys are case-sensitive; the route is declared with `:userId` but the controller reads `$routeParams.userId` — the casing mismatch means that exact key simply does not exist on the `$routeParams` object
- ☐ B) `$routeParams` must be manually populated via `$location.path()` before it becomes available
- ☐ C) `.otherwise()` is misconfigured and is intercepting the `/user/:userId` route before it can match
- ☐ D) Route parameters can only be accessed inside a route's `resolve` block, never directly inside a controller

Q25.

```
app.controller('WeatherCtrl', function($scope, $http) {  
  $http.get('/api/weather').success(function(data) {  
    $scope.temperature = data.temp;  
  });  
});
```

On AngularJS 1.6+, this throws `$http(...).success` is not a function. What is the bug and the correct modern fix?

- ☐ A) `.success()` and `.error()` were deprecated and then removed from `$http`'s returned promise starting in AngularJS 1.6; the fix is the standard promise API: `.then(function(response) { $scope.temperature = response.data.temp; })`
- ☐ B) `.success()` was never valid AngularJS syntax at any version; `$http.get` must always be followed immediately by `.then().catch()`
- ☐ C) The real bug is `data.temp` — it should be `data.data.temp`, since `$http` always double-wraps every response body
- ☐ D) `$http` requires an explicit `responseType: 'json'` configuration object as a mandatory second argument in every version

Q26.

```
<ul><li ng-repeat="tag in tags">{{tag}}</li></ul>

$scope.tags = ['urgent', 'bug', 'urgent', 'frontend'];
```

This throws [ngRepeat:dupes] Duplicates in a repeater are not allowed. What is the best fix that still preserves both 'urgent' entries in the rendered list?

- ☐ A) Remove duplicate values from `$scope.tags` before rendering, since AngularJS fundamentally cannot render duplicate primitive values under any configuration
- ☐ B) Use `ng-repeat="tag in tags track by $index"`, so AngularJS identifies each item by its array position instead of by value identity — duplicate primitive values can then safely coexist
- ☐ C) Wrap each string in `Object.freeze()` so AngularJS treats every tag as a unique reference
- ☐ D) Add a `key` attribute to the `<li>`, mirroring how React requires unique list keys

Q27.

```
app.controller('StatsCtrl', function($scope) {  
  $scope.stats = { views: 10, likes: 4 };  
  $scope.$watch('stats', function(newVal) {  
    newVal.ratio = newVal.likes / newVal.views;  
  }, true);  
});
```

This throws 10 \$digest() iterations reached. Aborting!. What causes the infinite loop, and what is a correct fix?

- ☐ A) The listener mutates the very object (`newVal.ratio = ...`) that the deep watch (the trailing `true`) is monitoring, so every digest iteration detects yet another change (the newly written `ratio` property) and reruns, never stabilizing; fix by computing `ratio` as a separate, non-watched property, or deriving it without writing back into the watched object
- ☐ B) `$watch` does not support object expressions; only string expressions referencing primitives are valid
- ☐ C) The third argument `true` is invalid syntax; it should be the string `'deep'`
- ☐ D) `stats.views` and `stats.likes` must each have their own separate `$watch`; watching an entire object directly is unsupported

Q28.

```
app.service('AuthService', function() {
  this.isLoggedIn = false;
  this.login = function() { this.isLoggedIn = true; };
  return { logout: function() { console.log('logged out'); } };
});

app.controller('NavCtrl', function($scope, AuthService) {
  AuthService.login();
  console.log(AuthService.isLoggedIn);
});
```

What happens when this code runs, and why?

- ☐ A) true is logged, because `.service()` always merges any returned object together with `this`
- ☐ B) `AuthService.login` does not exist, so calling it throws a `TypeError` before the `console.log` line is ever reached — when a `.service()` constructor function explicitly returns an object, standard JavaScript constructor semantics (which `$injector.instantiate` follows exactly) use that returned object instead of `this` as the actual service instance; the resulting `AuthService` is simply `{ logout: ... }`
- ☐ C) false is logged, because `this.isLoggedIn` silently resets after `.login()` completes
- ☐ D) true is logged, because JavaScript ignores explicit return statements inside any constructor function used by `.service()`

Q29.

```

app.directive('forceUppercase', function() {
  return {
    require: 'ngModel',
    link: function(scope, element, attrs, ngModel) {
      element.on('blur', function() {
        var upper = element.val().toUpperCase();
        element.val(upper); // directly manipulating the DOM
      });
    }
  };
});

```

After typing 'abc' and blurring the field, the input visually shows 'ABC', but `ngModel.$viewValue` and the bound model still hold 'abc'. What is the architectural bug?

- ☐ A) The directive manipulates the DOM element directly (`element.val(upper)`) instead of going through the injected `ngModel` controller, so AngularJS's model layer is never informed of the change; fix by calling `ngModel.$setViewValue(upper)` followed by `ngModel.$render()`
- ☐ B) `require: 'ngModel'` is missing its `^` prefix, so the directive cannot access the controller at all
- ☐ C) The blur event does not exist in AngularJS; only expressions bound via the `ng-blur` directive are supported
- ☐ D) `element.val()` is not a valid jQuery method and silently fails, which contradicts the visually-observed 'ABC' — so no real bug exists

Q30.

```

app.controller('SearchCtrl', function($scope, $http) {
  $scope.search = function(term) {
    $http.get('/api/search?q=' + term).then(function(response) {
      $scope.$apply(function() { $scope.results = response.data; });
    });
  };
});

<input ng-model="term" ng-change="search(term)">

```

This throws [`$rootScope:inprog`] `$apply` already in progress under certain conditions. What is the bug and the fix?

- A) An `$http` promise's `.then()` callback already executes inside a digest cycle that `$http` itself triggered internally to deliver the response; calling `$scope.$apply()` again inside that callback attempts to start a nested digest while one is already running — fix by deleting the manual `$scope.$apply()` wrapper and simply assigning `$scope.results = response.data;` directly
- B) `ng-change` can never be combined with any `$http` call
- C) `$scope.search` must be renamed, since `search` collides with a reserved AngularJS scope method name
- D) The real bug is the manual string concatenation in the URL; `$http.get` requires a `params` config object instead, and that mismatch is what triggers the error

Section D: Best Practice Questions (Q31–Q35)

Q31. For a large-scale AngularJS application spanning 50+ features (invoicing, inventory, HR, reporting, etc.), which folder-organization strategy is most widely regarded (e.g., in the well-known John Papa AngularJS Style Guide) as the more maintainable and scalable choice?

- A) Organize strictly by file type across the whole app — one top-level controllers/ folder for every controller in the entire application, one services/ folder for every service, one directives/ folder, and so on
- B) Organize by feature — each feature folder (invoicing/, inventory/, etc.) co-locates its own controllers, services, directives, and templates together, following the "LIFT" principle (Locate quickly, Identify at a glance, Flat structure, Try to stay DRY)
- C) Place every controller, service, and directive inside a single flat app.js file to minimize HTTP requests during development
- D) Organize by AngularJS version compatibility, keeping 1.2-compatible code separated from 1.5+ code in different folders

Q32.

```
app.controller('InvoiceCtrl', function($scope, $http) {  
  $scope.generateInvoice = function(orderId) {  
    $http.get('/api/orders/' + orderId).then(function(res) {  
      var tax = res.data.subtotal * 0.18;  
      var total = res.data.subtotal + tax - res.data.discount;  
      $scope.invoice = { subtotal: res.data.subtotal, tax: tax, total:  
total };  
      $http.post('/api/invoices', $scope.invoice);  
    });  
  };  
});
```

From a maintainability and testability standpoint, what is the primary architectural issue here, and what is the best-practice fix?

- A) There is no issue; controllers are meant to own both HTTP communication and business calculations like tax computation directly
- B) The controller mixes HTTP communication, business logic (tax/total calculation), and view-state assignment all in one function — making the tax logic impossible to unit-test in isolation and impossible to reuse elsewhere; best practice is to extract the HTTP calls and calculation logic into an injectable InvoiceService, leaving the controller to only orchestrate calls and expose data to the view
- C) The controller should be converted into a directive, since only directives are permitted to make \$http calls in AngularJS
- D) The only real issue is variable naming; renaming tax and total using Hungarian notation resolves the architectural concern

Q33. Profiling reveals that `$scope.$watch( 'largeDataSet' , function(newVal){ ... }, true)` — where `largeDataSet` is a deeply nested object with thousands of properties updated frequently — dominates digest-cycle time. What is the best-practice remediation?

- A) Increase AngularJS's digest TTL constant from 10 to a much higher number so the digest loop has more iterations available to stabilize
- B) Replace the deep watch with a narrower, targeted comparison (e.g., watching a version counter or a specific derived value that only changes when something meaningful actually changes), use `$watchCollection` where a shallow comparison suffices, or restructure the app to push updates explicitly instead of deep-comparing the whole object every digest — a deep (`true`) watch performs an expensive recursive equality check on every single digest cycle regardless of whether anything actually changed
- C) Wrap `largeDataSet` in `angular.copy()` before every digest so AngularJS can compare object references instead of deep values
- D) Deep watches are always $O(1)$ regardless of object size in AngularJS, so the slowdown must originate from an unrelated source

Q34. A static list of country names, loaded once at startup and never modified afterward, is rendered with `<li ng-repeat="c in countries">{{c.name}}</li>`. Profiling shows this binding contributes measurable, permanent digest-cycle overhead as the app's total watcher count grows elsewhere. What is the most idiomatic AngularJS fix?

- ☐ A) Replace `{{c.name}}` with `ng-bind="c.name"`, since `ng-bind` is inherently watcher-free
- ☐ B) Use one-time binding syntax — `{{::c.name}}` and `ng-repeat="c in ::countries"` — so AngularJS automatically de-registers the corresponding watcher(s) as soon as each bound value first becomes defined/stable, permanently removing them from every future digest cycle since the data is known never to change again
- ☐ C) Move the `ng-repeat` inside an `isolate-scope` directive, since isolate scopes are excluded from the digest cycle entirely
- ☐ D) Wrap the list in `ng-if="false"` immediately after the first render completes, which detaches its watchers while preserving the already-rendered DOM

Q35. A 40-person engineering team, building on a minifying bundler, occasionally ships Unknown provider production bugs when a new hire writes `app.controller('Foo', function($scope, SomeService) {...})` without array annotation. Which practice most effectively prevents this entire class of bug at scale, rather than relying on individual developer discipline?

- ☐ A) Add code-review comments reminding each developer, case by case, to manually annotate every function they personally write
- ☐ B) Integrate an automated build-time tool such as `ng-annotate` (or enforce a lint rule requiring `$inject`/inline-array syntax) into the build pipeline, so DI annotations are injected or validated automatically before minification, removing dependence on manual per-developer vigilance
- ☐ C) Disable minification entirely in production builds to sidestep the problem
- ☐ D) Require every service to be attached directly to the global window object so minification cannot rename any references to it

Section E: Advanced Concept Questions (Q36–Q40)

Q36. AngularJS's `$digest` loop reruns all watchers repeatedly until no watched value changes between two consecutive passes — but only up to a fixed maximum number of passes. What is that default maximum, what is it called internally, and what happens if it's exceeded?

- A) The default is 10 iterations, referred to internally as the digest TTL (Time To Live); if watchers keep reporting changes past the 10th pass, AngularJS throws `10 $digest() iterations reached. Aborting!` and abandons that digest entirely
- B) The default is unlimited by design, guaranteeing eventual consistency even if it means looping until the browser tab becomes unresponsive
- C) The default is 3 iterations, matching AngularJS's three lifecycle phases (compile, link, digest)
- D) The limit is configurable per-watcher via a `maxIterations` property passed as a fourth argument to `$watch`

Q37. Inside a deeply nested child controller, a developer calls `$scope.$digest()` directly (instead of `$scope.$apply()`) after an external async callback updates scope data. The bound expression in that same child view updates correctly, but a completely unrelated, separate top-level branch of the application — fed by an unrelated controller, whose own scope data also changed around the same time — does not update. What explains this?

- A) `$scope.$digest()` only processes the calling scope and its own descendant scopes downward — never upward to `$rootScope`, and never sideways into unrelated branches — whereas `$scope.$apply()` internally invokes `$rootScope.$digest()`, which walks the entire scope tree from the root down, reaching every branch including unrelated siblings
- B) `$digest()` and `$apply()` are functionally identical; the sibling branch's failure to update must stem from an unrelated bug elsewhere
- C) `$digest()` only operates on isolate scopes, never on normally-inherited scopes, which is why only the local child view updated
- D) Sibling branches never update from any digest unless `$broadcast` is explicitly called on `$rootScope` first

Q38.

```

app.directive('logA', function() {
  return { restrict: 'A', priority: 100, link: function(){ console.log('A
link'); } };
});
app.directive('logB', function() {
  return { restrict: 'A', priority: 200, link: function(){ console.log('B
link'); } };
});

<div log-a log-b></div>

```

Both directives are non-terminal and rely on default (postLink-only) linking, and both are declared on the same element. What determines execution order here, and what is the resulting console output?

- ☐ A) Declaration order in the HTML attribute list (log-a appears before log-b), so 'A link' logs before 'B link'
- ☐ B) Directive priority determines execution order among directives sharing one element — higher-priority directives compile and link before lower-priority ones — so since logB's priority (200) is higher than logA's (100), 'B link' logs before 'A link'
- ☐ C) Directives on the same element always execute in alphabetical order of their normalized name, so 'A link' logs first regardless of priority
- ☐ D) AngularJS throws a compile-time error whenever two directives with different priorities are placed on the same element without an explicit `terminal: true` flag

Q39. A team adds `ng-strict-di` to their root element (`<html ng-app="myApp" ng-strict-di>`). Afterward, several previously-working controllers throw errors resembling: `$injector:strictdi - function is not using explicit annotation and cannot be invoked in strict mode`. What does `ng-strict-di` actually do, and why did these errors surface?

- A) It disables `$http` caching, and the new errors are unrelated cache-invalidation issues that coincidentally appeared at the same time
- B) It forces AngularJS to invoke controller/service/component constructor functions without falling back to the reflection-based implicit annotation (which parses parameter names via `Function.prototype.toString()`); any function relying on implicit annotation instead of the `$inject` property or inline-array syntax now fails immediately in development — deliberately surfacing minification-unsafe code before it ever reaches a minified production build
- C) It automatically minifies the application in development mode to preview production behavior, and these errors are artifacts of that preview minification
- D) It requires every `$scope` variable to carry an explicit TypeScript-style type annotation written as a comment

Q40. A page manually bootstraps two separate AngularJS applications via two calls — `angular.bootstrap(element1, ['appA'])` and `angular.bootstrap(element2, ['appA'])` — on two different DOM regions, both loading the identical module `appA`, which registers a `.factory('CounterService', ...)` wrapping an internal counter starting at 0. Code in the first region calls `CounterService.increment()`. Does the second region's injected `CounterService` reflect that change?

- ☐ A) Yes, always — factories registered under the same module name are globally deduplicated by AngularJS across the entire page, no matter how many times the app is bootstrapped
- ☐ B) No — each call to `angular.bootstrap()` creates its own, entirely independent `$injector`, and singleton services are singletons only within a single injector; two separate injectors, even when loading the exact same module, produce two completely separate `CounterService` instances with independently-starting state
- ☐ C) It depends on whether `$rootScope` was manually shared between the two bootstrap calls using `angular.element().scope()`
- ☐ D) Yes, but only if both DOM regions are wrapped in elements sharing the identical `ng-app` attribute string value